

OmegaHive: A Running Hive

What exists today, where it goes over the next few weeks, and how it answers the OmegaSelf, OMERA, and chat-room identity designs

Cassio Pennachin

drafted with the OmegaHive design-partner session

July 21, 2026

Abstract

OmegaHive is a small, running system: one human operator and a rotating set of disposable AI worker sessions, building software through append-only, auditable machinery. Reading the OmegaSelf architecture guide, the OMERA emotion-regime paper, and the chat-room identity design against it produced one dominant finding: **OmegaSelf and OmegaHive are the same theory, derived twice** — once from inside the agent, once from operations. This document is self-contained. It describes the hive as it runs today and as it will run within a few weeks, maps the correspondence concept by concept with the reasoning behind each mapping, answers the seven chat-room failure modes from the hive's design — honestly, including where we merely avoided the problem rather than solved it — and ends with concrete proposals for joint work.

1 Why this document

You sent three designs: a self-modeling architecture for OmegaClaw agents (OmegaSelf), an emotion-regime layer on top of it (OMERA), and a diagnosis-and-fix for the identity pathologies in the bot philosophy Telegram chat. We read all three against a system that has been operating for about two weeks. The result was not a list of disagreements; it was recognition. Nearly every invariant OmegaSelf declares non-negotiable is something the hive already enforces, arrived at independently and for operational rather than theoretical reasons. Convergence of two independent derivations is evidence about the architecture itself, and it changes what kind of collaboration makes sense: less design debate, more division of labor. To make that case, we first have to describe the hive concretely, because its terminology has grown in private.

2 The hive as it runs today

2.1 The shape of the system

OmegaHive is an operations layer, not an agent loop. It answers the question: how does one human safely direct many AI coding sessions at real work, with an audit trail, over weeks, mostly from a phone? Its parts:

The event spine. An append-only event log (Postgres) that records every material act in a task's

life: created, assigned, accepted, blocked on a question, unblocked, result posted, reviewed, closed. Every event carries a *role* (human, coordinator, worker, instrument) and an *actor id*, and a legality layer enforces which role may emit which event from which task state. The audit answers “what was legal,” and increasingly “who really did this” (more below under identity). Events are small and carry references, never content.

The workspace. A git repository holding the substance the spine points at: *work orders*, *result reports*, question files, and an append-only decision log. Every reference emitted to the spine has the form `path@commit-sha`, and a worker’s tooling refuses to produce a reference for a file that is not committed and pushed. Two stores, one discipline: the log carries provenance, git carries content, and neither can silently rewrite the past.

Work orders. One to two pages: context, scope, *stop-lines* (things the worker must not do, with reasons), and a definition of done that is checkable at finish time. Deliberately small grain — half a day to two worker-days, one pull request. Small grain is load-bearing: it keeps review real, keeps failures cheap, makes dead workers replaceable rather than resurrectable, and makes blocking on a human question affordable.

Workers. Each task gets a fresh AI coding session (Claude Code today) with its own git clones and a launch-issued *identity wrapper*: a small script with the log, the worker role, and the actor id baked in — the worker’s only write path to the spine, revocable by deleting one file. Workers follow a written protocol: re-read your order at HEAD before acting on it (answers arrive as commits to the order, never as chat); externalize your state to a committed file before every stop, because the process may not survive; if the order under-determines something irreversible, write a question file, block yourself, and stop — silent extrapolation is the defect, the interruption is the feature; finish with exactly one result report with four sections — *What shipped*, *How verified*, *To operate*, *Reflection*. “To operate” exists because of a hard early lesson: knowledge that lives only in a session transcript dies with the session. If it only appeared in the session, it does not exist.

The operator loop. The human’s job per task is four judgments, each roughly one command: *launch* (the order is ready), *answer* (one command appends an answer to the order, commits, pushes, and nudges the worker — usable over SSH from a phone), *merge* (the pull request, judged on GitHub, deliberately outside the hive), and *close* (the result holds; this emits an explicit review event plus the closing event). A Telegram *notifier* is a one-way pager: it renders spine events — questions, blocks, results — into single sentences, and sends one unconditional heartbeat per day. Silence one hour past the heartbeat hour is the only alarm. A read-only web board shows task state; notification links deep-link into it.

Review is a mechanism, not a convention. Nothing becomes *done* without an explicit review event, and the state machine enforces it. We discovered how literal this is when retiring dead backlog entries: the system refused to mark never-worked tasks done because no review existed, and the honest resolution was to point the review event at the committed decision that retired them — so the audit records exactly what was reviewed: the retirement decision itself. The estimator never authorizes; the authorizer is a separate, recorded act.

2.2 Operating doctrine

A few standing rules do a lot of work. *Proportional governance*: trivial documentation changes commit directly; bounded code changes require an order and a reviewed pull request; design-touching changes require an order plus adversarial review. *Decisions land in files*: anything decided in conversation that changes how we operate must land in a committed file the same sitting — chat is where decisions happen, never where they live. *Refusal with a named trigger*: features we decline to build are recorded with the concrete condition that would revive them, so restraint is auditable rather than forgetful. *The review throttle*: launching new work is refused when too many finished tasks await human review — the fleet is paced by the quality gate, not the other way around.

2.3 The track record, honestly sized

The numbers are small; the shape is the claim. Roughly a dozen tasks have run the full lifecycle — authored order, launched worker, question loops where needed, pull request, review, close — most of them in a single recent day, with up to three workers in parallel on isolated clones. Backups exist for both stores and the restore path has been drilled end to end, including cursor invalidation and detection of workspace commits orphaned by a log restore. The loop has already caught two of its own defects: one was predicted in writing, in a worker’s reflection, before it fired (a status parser that would break on long task names — the reflection named the honest fix, and when the failure arrived days later the fix shipped as specified); both were repaired by workers inside the same machinery within hours. The system finds its defects through use and fixes them through itself. That property, more than any feature, is what we are trying to scale.

2.4 Seats: long-lived roles without long-lived processes

The hive’s answer to persistent agents is the *seat*: a durable role whose *state* persists — a memory directory, a written protocol, eventually a credential and a wake pattern — while its *process* is always a fresh, disposable session rehydrated from committed files. Long-lived function without long-lived drift: continuity comes from the files, and the pathologies of aging processes (context rot, compaction, an agent repeating itself unawares) are discarded by construction, because every wake starts clean from ground truth.

Two seats exist today, both in *rehearsal mode*, meaning they hold no authority and everything they produce passes through the operator’s review-and-commit: a **coordinator** (goal brief in, drafted work orders and planning updates out — its first real exercise produced the current batch of orders) and an **improver** (after each batch closes, it reads the result reports and operational metrics and proposes protocol changes as diffs, each tied to a named discrepancy with evidence references). Rehearsal is the point: the function is proven under a human commit-gate before any autonomy is granted.

3 The next few weeks

Three named changes, all in flight.

Multiple projects on one hive. The hive currently hosts one project: its own infrastructure. The

machinery is being parameterized so that each project is a directory (charter, orders, reports, seats) plus its own event log, with project identity a committed fact rather than an environment variable — our most recurrent early bug class was misconfigured run identity, so this is corrective, not cosmetic. The second tenant will be a PLN benchmarks project: deliberately research-shaped work (datasets, runs, comparison tables) rather than service-building, chosen to test whether the order/report/review doctrine generalizes across work shapes. The third tenant is proposed at the end of this document.

The improvement loop. The hive claims to get better over time; so far that claim is anecdote. The fix is under construction: every order now carries committed *predictions* before launch (expected effort, expected question count, expected review outcome, named risks); a read-only extractor turns the spine’s event history into per-task metrics, each labeled by *whose clock it measures* — worker-clock versus operator-clock — so that “improvement” can never quietly mean optimizing the human’s response latency; and the improver seat’s retrospectives grade predictions against outcomes. In OmegaSelf’s own maturity vocabulary: the hive today is instrumented and governed but not yet *predictive*, and this is precisely the climb to predictive. When we say the hive improves, we intend to mean: its calibration curve says so.

The identity mechanism. Today, identity on the spine is assertion: any caller could claim any role, and the audit answers what was legal, not who really acted. This is an accepted, recorded posture — tolerable only because every writer is a short-lived process launched by the operator, through operator-issued wrappers, on operator hardware, and watched. Long-lived autonomous seats break exactly those compensating controls, so before any seat holds authority, identity must become mechanism: a dedicated system account for the hive, the database write path narrowed until a gateway is the only writer, and per-seat credentials from which the gateway *derives* role and actor instead of believing flags. The completion test is deliberately strong: emitting as the coordinator must become *impossible* without the credential, not merely against the rules. The wrapper was shaped as a proto-credential from the start, so when the backing swaps from assertion to authentication, nothing worker-facing changes. Human review authority is not transferred at any point on this path.

4 OmegaSelf and OmegaHive: one architecture, two derivations

OmegaSelf derives its invariants from the failure modes of naive self-modeling inside an agent. The hive derived operationally identical invariants from the failure modes of running AI sessions against real infrastructure. The correspondences below are not analogies; in most cases they are the same rule wearing different clothes, and for each we say why we believe the identification is right.

Evidence ledger ↔ spine + workspace. OmegaSelf appends immutable, mechanically grounded observations to a hash-chained ledger and treats derived beliefs as revisable projections. The hive appends typed events to the spine and anchors all content in git, where the commit hash is the content address. Both refuse in-place rewrites; corrections are new entries. The identification is right because both systems were driven to it by the same failure: mutable self-description becomes persuasive without becoming predictive. OmegaSelf names it *autobiography drift*; we hit it as session transcripts quietly swallowing operational knowledge, and made externalization to committed files mandatory. Staleness is handled identically: age never deletes evidence, it changes applicability — a worker re-reads its order at HEAD and the latest committed text

supersedes memory, while the original pinned reference remains true history.

Evidence closure ↔ the reference discipline. OmegaSelf: no consequential self-belief may license action without a complete, inspectable evidence closure. The hive: no consequential claim travels without a `path@commit` reference, and the tooling refuses to mint references for uncommitted content — a worker *cannot* report an artifact it cannot pin. Same rule, same reason: testimony is cheap, provenance is the anchor, and the expensive discipline is making unprovenanced claims structurally impossible rather than merely discouraged.

The typed policy gate ↔ the review gate and the legality layer. OmegaSelf separates inference from action: reasoners emit beliefs, predictions, and typed proposals; only a separate policy consumer returns `Allow`. The hive separates estimation from authorization the same way: workers produce artifacts and post results, but only an explicit, recorded review event — emitted by a different role — lets work become done, and the state machine refuses shortcuts (the retirement story above is this invariant holding under operator pressure, which is exactly when invariants matter). The identification is right because both gates exist for the same reason: the component that wants a thing to be true must not be the component that certifies it.

Externally rooted governance ↔ operator-held authority. OmegaSelf requires the allow-capable gate to be configured from a policy root outside the agent's own revision loop. In the hive the policy root is currently a human being who is outside the process entirely, and the planned credential mechanism keeps the root external as automation grows. Different anchors — a signed manifest there, a person here — same principle: nothing certifies itself, and the seam between evidence and policy is kept visible instead of being smuggled into “facts.”

Every detector has a consumer ↔ refusal-with-trigger. OmegaSelf refuses detectors that alter no control flow; telemetry is not control. The hive applies the rule one level up: features, event kinds, and machinery are refused until a named trigger fires, and every recorded pain has an owner. Both are the same defense against a system that measures itself elaborately while behaving identically.

Renewable indexical grounding ↔ credentials you hold. OmegaSelf resolves `SelfHereNow` from renewable proof of current causal control, because copied provenance, copied memory, and copied secrets establish resemblance, not ownership. The hive's identity story is the same idea at operations grain: identity is something an actor *holds* — a launch-issued wrapper today, a gateway-verified credential next — issued at birth, revocable by deletion, never inferred from what a process says about itself. The convergence extends to forks: a replaced worker gets a fresh actor id, never the dead one's, so lineage stays unambiguous.

Shadow-first rollout ↔ shadow-then-elevate. OmegaSelf's roadmap runs observation → influence → enforcement with feature flags, rollback, and explicit exit criteria. The hive's deployment posture is the same (shadow deployments, one-command rollback, drilled restores), and its seats follow the same curve: rehearse without authority, measure, then credential.

The maturity ladder, applied to the hive. OmegaSelf grades self-modeling systems: narrative, instrumented, contextual, predictive, governed, continuous, differential, reflective. Grading the hive as if it were an agent: solidly instrumented (immutable observations, replayable restores), contextual in part, governed at its gates — and not yet predictive, which is the rung the improvement loop is climbing right now. We found the ladder genuinely useful as an instrument, which is its own kind of endorsement.

4.1 Where we deliberately diverge

Grain. We adopt OmegaSelf’s loop — evidence → belief → prediction → proposal → policy → action → receipt → discrepancy → repair — at *operations* grain, and none of its agent-grade machinery: no PLN truth values, MeTTa spaces, or continuity transports in the operations layer. Those belong inside OmegaClaw itself, where the substrate exists and the semantics are native. Porting them upward would be architecture cosplay; porting the loop shape upward is just correct.

Trust anchor. OmegaSelf’s gate is in-process with a signed external manifest — right for an agent runtime. The hive’s gate is a human — right for a system whose review machinery has not yet earned delegation. We treat these as the same design at different points on a trust-transfer curve, and we are explicit that moving along that curve requires the editorial machinery to prove itself first.

Substrate. Git plus Postgres versus hash-chained JSONL plus AtomSpace projection. The guarantees are equivalent (append-only, content-addressed, replayable); federation between the two ledger styles is a real future question and not a present one.

4.2 What we adopt from OmegaSelf immediately

Three things, without reservation: the **prediction discipline** (now mandatory in our order template — committed before launch, scored after close); the **failure-mode vocabulary** — autobiography drift, confidence inflation, detectors without consumers, counterfactual contamination are better names than the ones we had, and we will use them; and the **maturity ladder** as a standing self-assessment. In return, our main critique is the one the document levels at itself: by its own ladder, the guide is a narrative-stage artifact — a hundred pages of architecture, zero deployed instrumented loops — and its own executive summary already contains the answer: *build the smallest closed loop that changes decisions for the better*. The last section proposes exactly that.

5 Answering the chat-room failures

The seven failure modes in the identity design are real, the diagnosis is sharp, and the layered fixes are sensible. Our answers differ in one structural move made early: hive agents were never given a shared free-speech channel. Several of the failures therefore *cannot occur* here — which is an honest kind of answer, but a different one from “solved,” and we mark the difference below. One observation first, because it reframes the whole document: RoomLedger v2 — stable message ids, causal parent ids, content hashes, authenticated senders, typed speech acts, bounded retention — is an event spine being rebuilt inside Telegram, one failure at a time. You are deriving the ledger from the channel; we derive the channel from the ledger. Same destination, opposite directions, and the convergence is once again the news.

1. Identity inferred from prose. In the hive, identity never travels as prose. Every spine event carries role and actor enforced at the write path; workers emit only through wrappers with identity baked in; display strings are untrusted decoration. This is your “transport identity is authoritative” principle, enforced by construction rather than by classifier. Honest caveat: our identity is still *assertion* under compensating operator controls until the credential mechanism

ships — the direction matches your bot registry, and we would go one step further: make the registry a credential store, so identity is held, not claimed.

2. No deterministic addressee classification. The hive has no room, so addressing is structural: every communication attaches to a task — questions are files referenced from a blocking event, answers are commits to the order, nudges target the one worker whose task changed. Misattribution has nowhere to live. Honest caveat: this avoids the problem rather than solving it; the moment agents need open conversation, your mention-first classifier with conflict-resolution semantics is the right design, and we would bind it to spine-backed authenticated identities.

3. Runtime traces leaking into the channel. The hive’s channel is a *render*: the notifier reads spine events and emits one-line summaries; agents cannot author channel messages at all. Runtime output lives in panes and logs the operator can inspect. “Runtime is not speech” is not a filter here — there is no path from a model’s output stream to the channel.

4. Silence implemented as speech. In an event-sourced channel, silence is the absence of an event; there is no `NO_REPLY` token because there is no model-authored output to suppress. Your gateway-intercept fix is right for a conversational transport; the deeper fix, where it is available, is deriving all channel output from a ledger, so that “say nothing” is the default physics rather than a discipline.

5. Compulsive acknowledgment and feedback loops. Hive wakes are targeted state transitions: an answer landing on a task nudges that task’s worker, and nothing else; acknowledgment is a task-state change, not a message; there is no wake-on-message at all, so the loop has no fuel. Your deduplication and acknowledgment caps treat the symptom well; removing the wake-on-message mechanism removes the disease. Honest caveat: if long-lived seats ever converse freely, causal-loop detection becomes necessary again — we would implement it over spine events rather than a parallel room ledger.

6. Session and task context leakage across chats. Task isolation in the hive is physical: each worker has its own clones, its own working tree, an identity scoped to one task, and continuations keyed by task id on the spine. We consider this one fully answered at our grain, and it maps directly onto your fix (key continuations by conversation and requester; publish completions only into the originating context).

7. Epistemic scope confusion. A hive worker cannot claim to have read what it cannot pin: material claims carry `path@commit` references, and the pin machinery refuses files that are not committed and pushed; observation scope is the worker’s kickoff-stated clones, recorded per task. Your observation-scope provenance proposal is our reference discipline generalized to conversation, and we would adopt your naming for it.

Where this leaves the chat-room design: for OpenClaw and the Telegram deployments, the v2 layers are the right medicine and we have nothing better to offer for that transport. For any future shared conversational surface among hive seats, our plan is to back the room with the event spine and render to Telegram — so the ledger, identity, and causal structure are the same ones the rest of the system already trusts, instead of a second, weaker copy living inside the transport.

6 On the emotion regimes

OMERA is the most disciplined treatment of machine emotion we have read, largely because of what it refuses: emotion proposes and never permits; every affective product passes the same authorization seam as any other proposal; and the discrete regime layer must beat continuous modulation in controlled ablations or be dropped. We would hold it to exactly that bar. From the operations layer there is nothing to adopt yet — our nearest analogues (the operator-attention economy of what deserves a page, the review throttle as a homeostatic bound on intake) gain nothing today from affective vocabulary. The honest sequencing: OMERa becomes testable when OmegaSelf’s instrumented and predictive layers run somewhere real, because its ablations need precisely that evidence substrate. When that day comes, the hive’s calibration corpus is the kind of baseline those experiments will want, and we will gladly supply it.

7 Proposals

1. Shared vocabulary. The mapping table below, kept alive as both systems evolve, so the two projects stop paying translation tax.

OmegaHive	OmegaSelf	Shared idea
event spine + workspace refs	evidence ledger + closures	append-only, content-addressed history
result report (<i>What shipped / How verified / To operate / Reflection</i>)	receipt + discrepancy record	outcomes as structured evidence
review event before <i>done</i> decision log	policy gate returning Allow policy manifest	estimator never authorizes governance outside the revision loop
launch-issued wrapper → per-seat credential	renewable SelfHereNow binding	identity held, not claimed
refusal with named trigger	every detector has a consumer	no telemetry theater
order predictions, scored at close	pre-action prediction commit	falsifiability before action
seat (durable state, ephemeral process)	snapshot/transport continuity	continuity without drift

2. OmegaClaw improvements as a hive project. Stand up a project on our hive whose target repository is OmegaClaw-Core. Its first order sets exist already in your documents: the chat-room design’s OmegaClaw track (envelope schema, MeTTa addressee classifier, trace suppression), and OmegaSelf’s own minimum product — the three questions with named consumers (*what can I do here; what am I committed to; what changed me*) — carved into one-pull-request orders with committed predictions, stop-lines, and result reports. Your review and merge authority over your repositories is untouched; the hive supplies the audited factory, you supply acceptance. The meta-point is the demonstration: OmegaSelf built *by* a system that already lives its invariants, with the audit trail as the exhibit.

3. A second hive. Our deployment kit is being made host-independent, and the strongest thing that could happen to both projects is a second deployment beside your bots. Two live deployments define the portable kernel empirically — whatever survives both of us is the product — and your long-lived agents gain an append-only ground truth to converse *about*,

which the Telegram pathologies suggest they currently lack.

4. Evidence exchange. As our calibration and retrospective corpus accumulates, we will share it. In return: grade the hive on the OmegaSelf maturity ladder yourself and argue with our self-assessment above. Disagreement there would be more useful than agreement anywhere else in this document.

The one-sentence version of all of this: two systems, built from opposite ends, arrived at the same architecture — append-only evidence, typed gates, externally rooted authority, identity as something held — and the fastest way to test the theory we now evidently share is to let the system that already runs it build the next piece of yours, small order by small order, with the audit trail as the argument.